

TIES-MERGING: Resolving Interference When Merging Models

Prateek Yadav¹ Derek Tam¹

Leshem Choshen^{2,3} Colin Raffel¹ Mohit Bansal¹

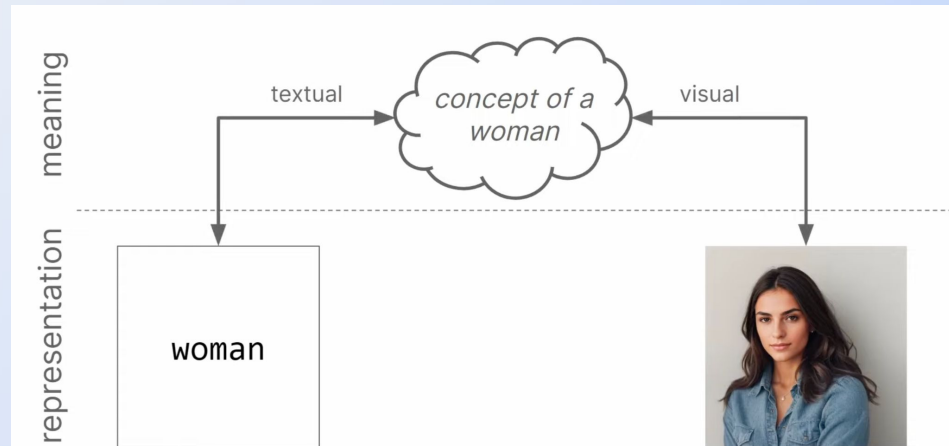
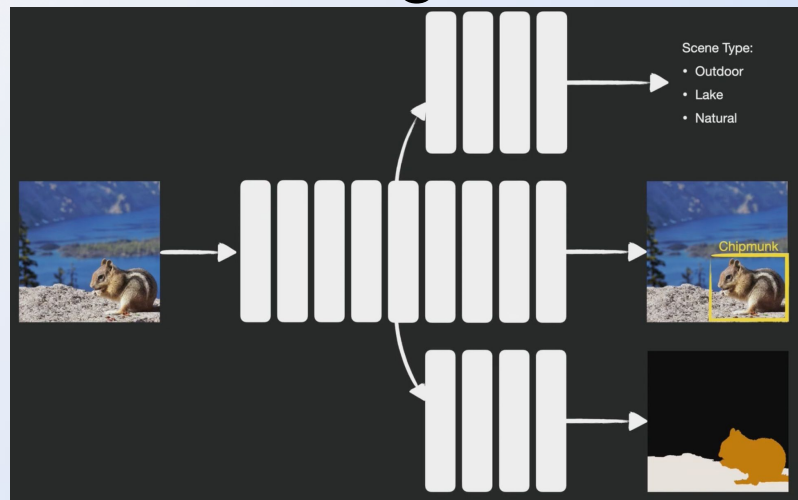
¹ University of North Carolina at Chapel Hill ² IBM Research leshem.choshen@ibm.com
{praty,dtredsox,craffel,mbansal}@cs.unc.edu

MLBBQ

4/24/26

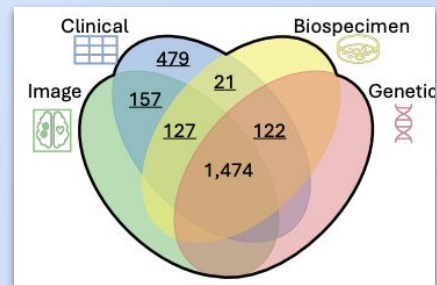
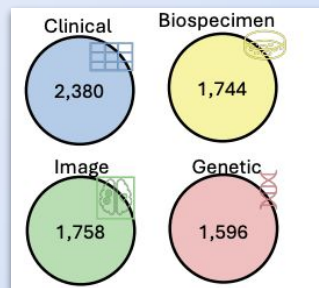
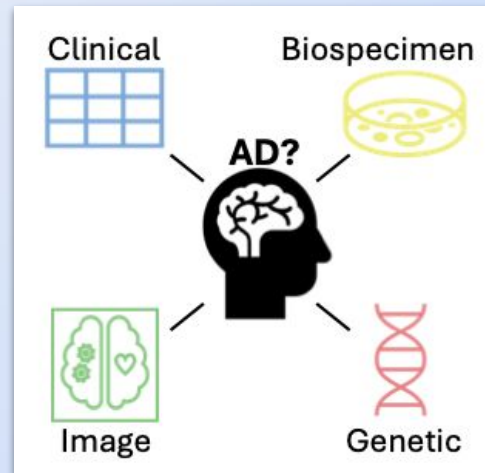
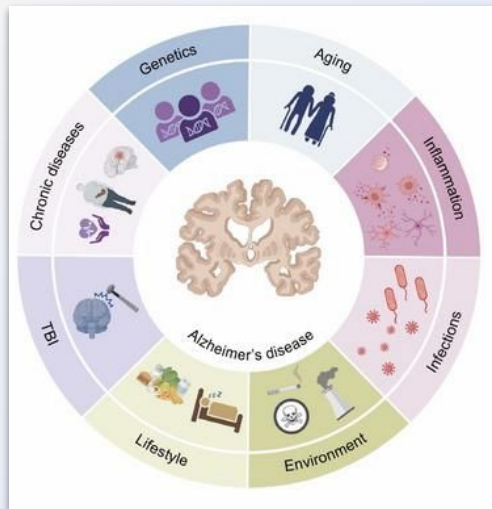
Multitask, Multiview, Multimodal Learning

- **Multitask learning:** Train a model to perform multiple tasks at once
 - E.g., classification, segmentation, and object detection
- **Multiview learning:** Train a model to operate on multiple representations of the same underlying concept
 - E.g., text and images of a woman
- **Multimodal learning:** Train a model to operate on multiple distinct sensor types or data formats that capture fundamentally different kinds of information



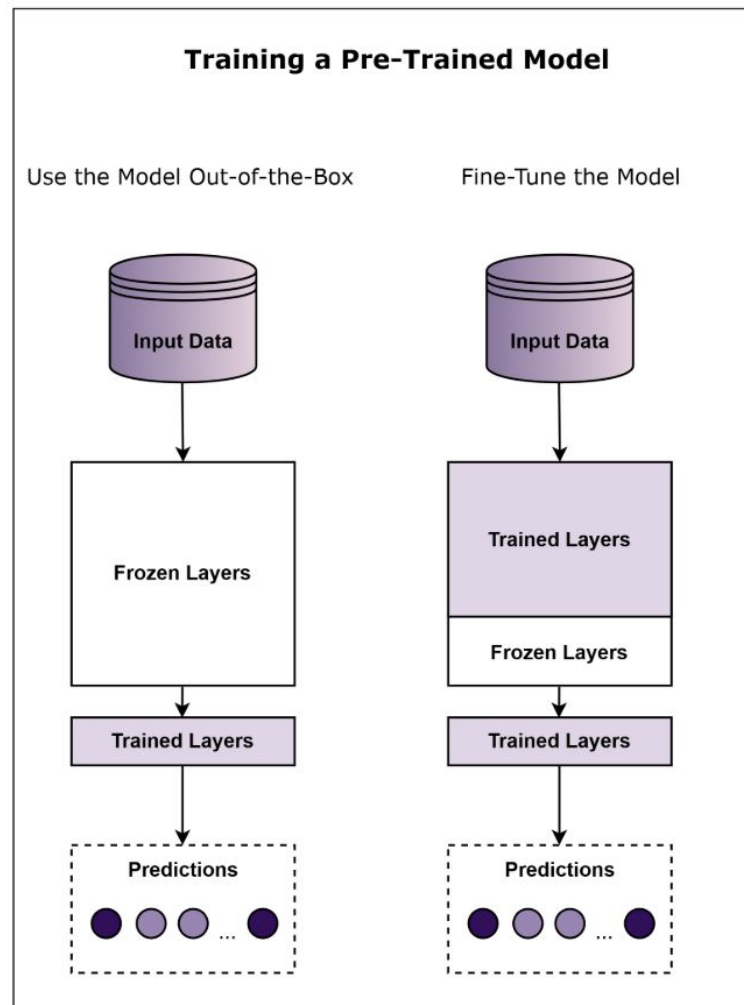
Complex Pathologies & Deep Learning

- **Multiview:** learning problem
 - How to best use similarities and differences of each view of the brain
- **Multitask:** learning point of view
 - Each measurement offers its own learning signal, the goal is how to best utilize that
- **Multimodal:** structure of data
 - How to respect the heterogeneity of the data with a modality and/or task agnostic model



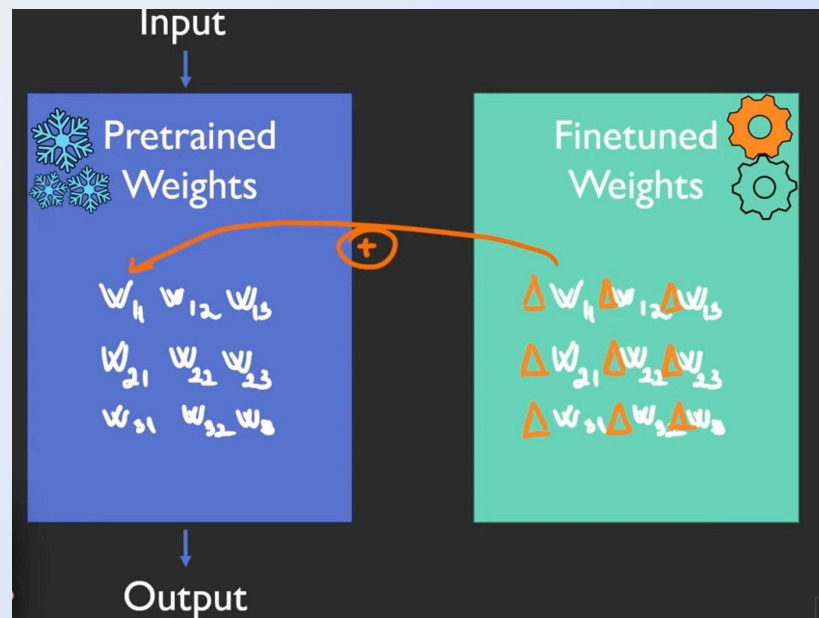
Pretrained Models

- Models previously trained on large, diverse datasets to perform specific tasks, which can then be reused or refined for new, related tasks.
- E.g., trained on one dataset(s)/task, then weights are frozen. This model can be used as is or fine tuned to a different task.
- Foundation model: a pretrained model trained on massive amounts of data designed to serve as a general base model to be adapted to other tasks.



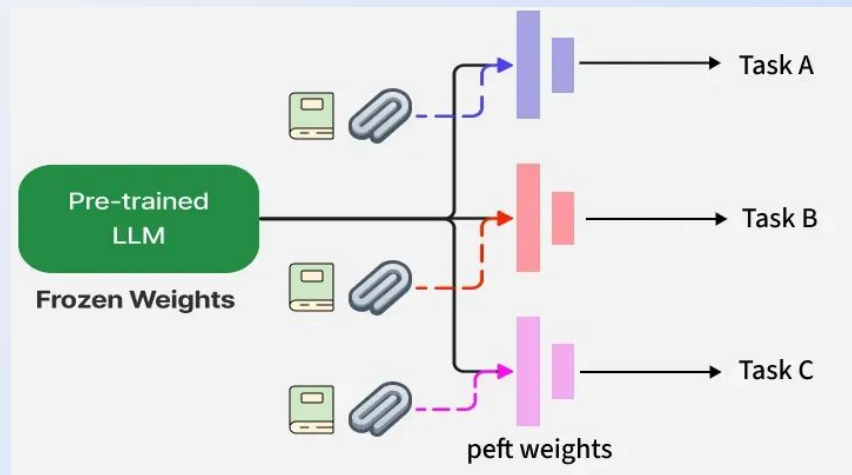
Fine Tuned Models

- FTMs are pretrained models further trained on specific datasets to improve performance on specialized tasks.
- Subtracting the weights of the pretrained model from the fine tuned model gives you delta weights.
- Delta weights represent all the change that has happened in fine tuning, incorporating the additional ability of the fine tuned model. **AKA task vectors.**



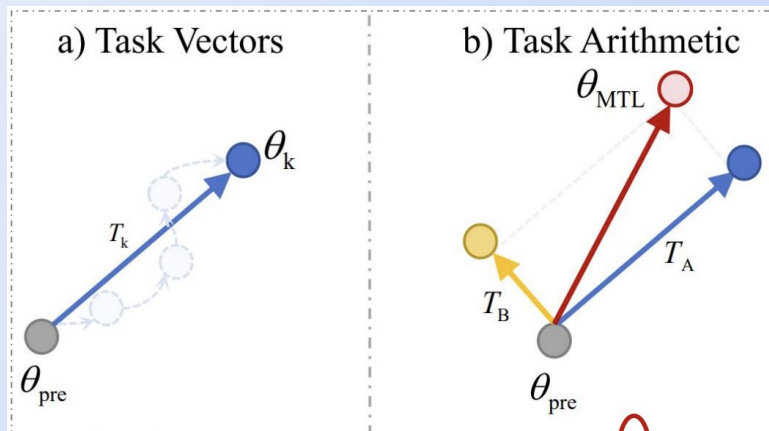
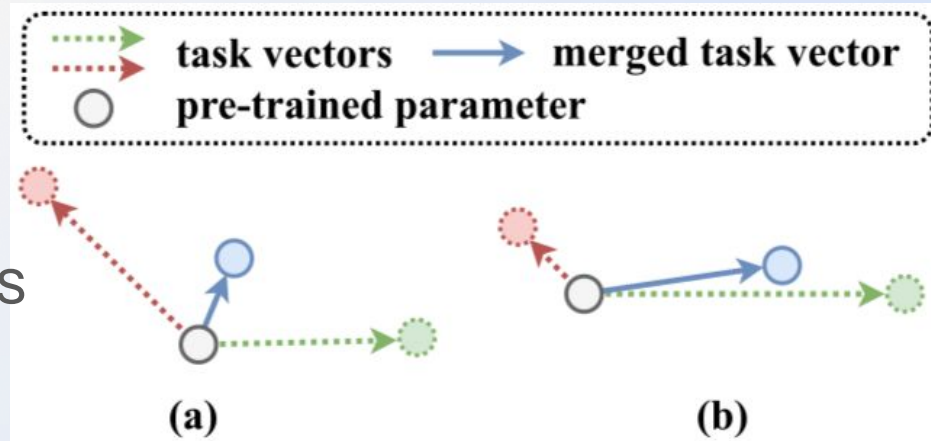
Parameter Efficient Fine Tuning (PEFT)

- **PEFT**- only train a small number of additional parameters while keeping the base model frozen.
- **(IA)³**- the specific PEFT method used in this paper. "Infused Adapter by Inhibiting and Amplifying Inner Activations."
 - Inserts learned scaling vectors at certain points in the network that rescale the activations.



Merging Models

- Goal is to merge the task vectors of pretrained models from different tasks.
- If merged successfully, then you are able to get a multimodal model while avoiding joint multitask training.



Challenges in Merging

- Most changes during fine tuning are not meaningful and are also extremely small
- Signs of the delta weights of multiple models can be opposite and when combined, lead to degradation
 - Positive delta weight: fine tuning increased this value
 - Negative delta weight: fine tuning decreased this value
 - Sign conflicts increase w/ # of models

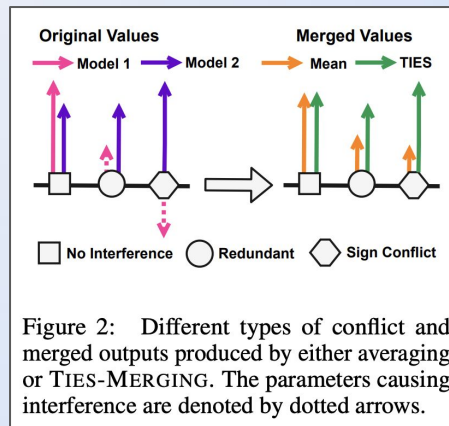
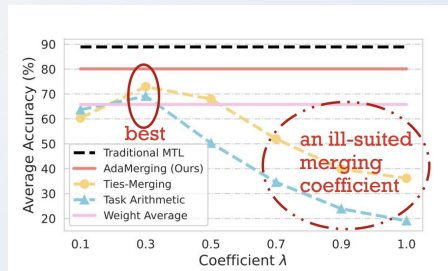


Figure 2: Different types of conflict and merged outputs produced by either averaging or TIES-MERGING. The parameters causing interference are denoted by dotted arrows.

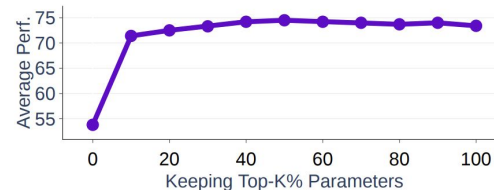


Figure 3: **Performance depends on a small fraction of high-magnitude parameters.** For each task vector, we keep only the largest - top- k % parameters and plot the average performance across eleven tasks. Keeping only the top-20% of the parameter does not degrade the performance.

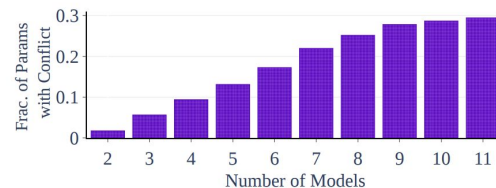


Figure 4: **Sign conflicts occur even after trimming and increase with the number of models.** We plot the fraction of parameters that have a sign conflict after trimming versus the number of models being merged.

Model Merging vs Joint Training

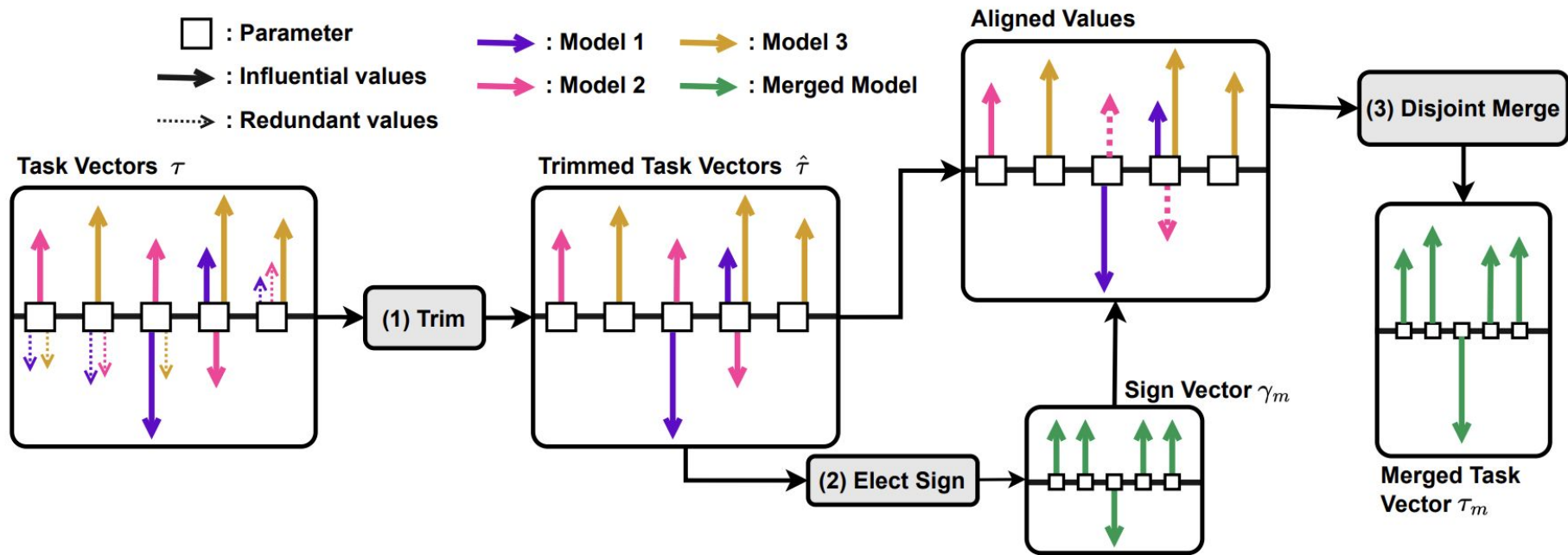
Model Merging

- Pros
 - Modular
 - No negative transfer during training
 - Can merge existing FTM's
- Cons
 - Lossy
 - Performance lagging
 - Sign conflicts
 - Assumed shared initialization

Joint Training

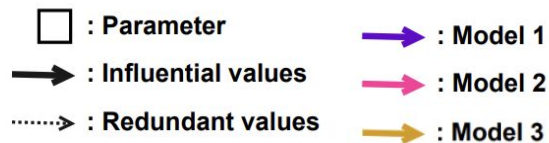
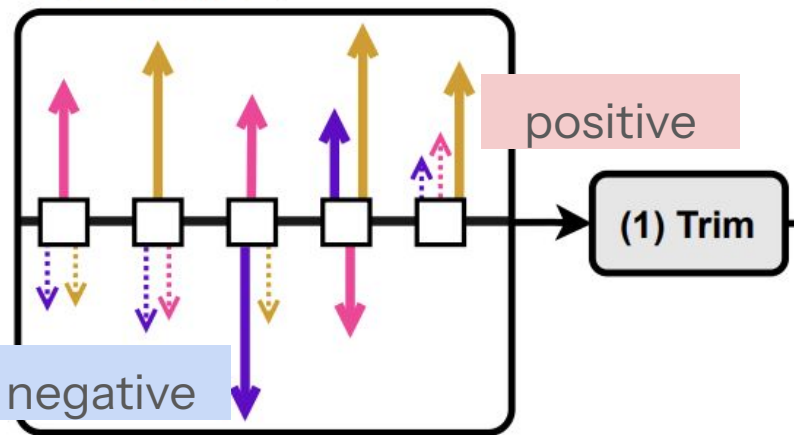
- Pros
 - Model sees all tasks simultaneously
 - Can learn to balance tasks
 - No merging needed
- Cons
 - Requires all data at once
 - Negative transfer
 - Computationally more expensive
 - Loss balancing

TIES Merging: Trlm, Elect



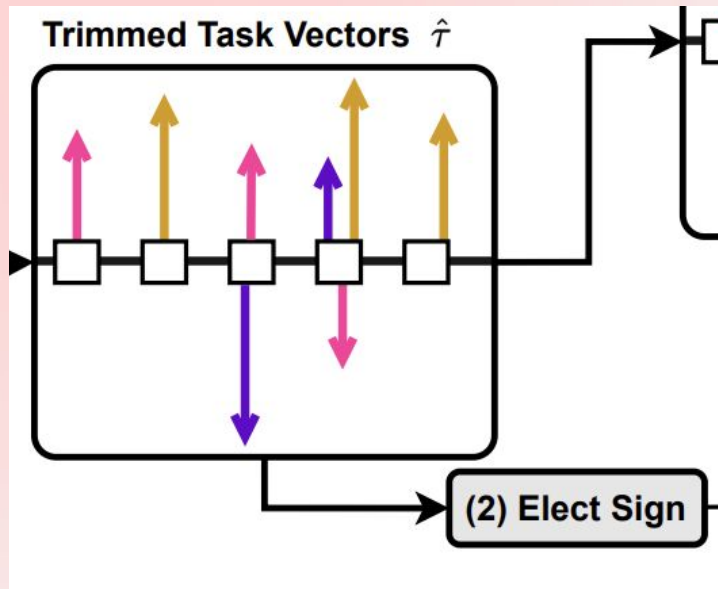
TIES Merging: Trim

Task Vectors τ

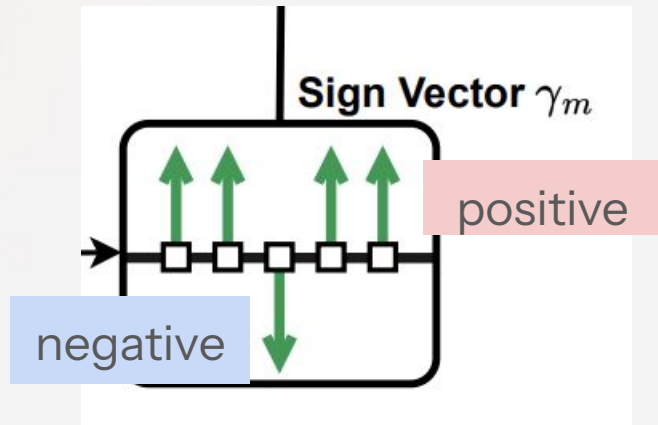


- For each parameter we might have model task vectors with either positive or negative sign
 - Magnitude denoted by arrow length
 - Sign is either top (pos) or bottom (neg)
 - Each model has its own color in this figure
- Dotted lines represent task vectors that can be set to zero without affecting model performance
- Trim step: remove redundant weight values
 - Keep top k% of highest mag. task vectors
 - Set others to zero

TIES Merging: Elect Sign

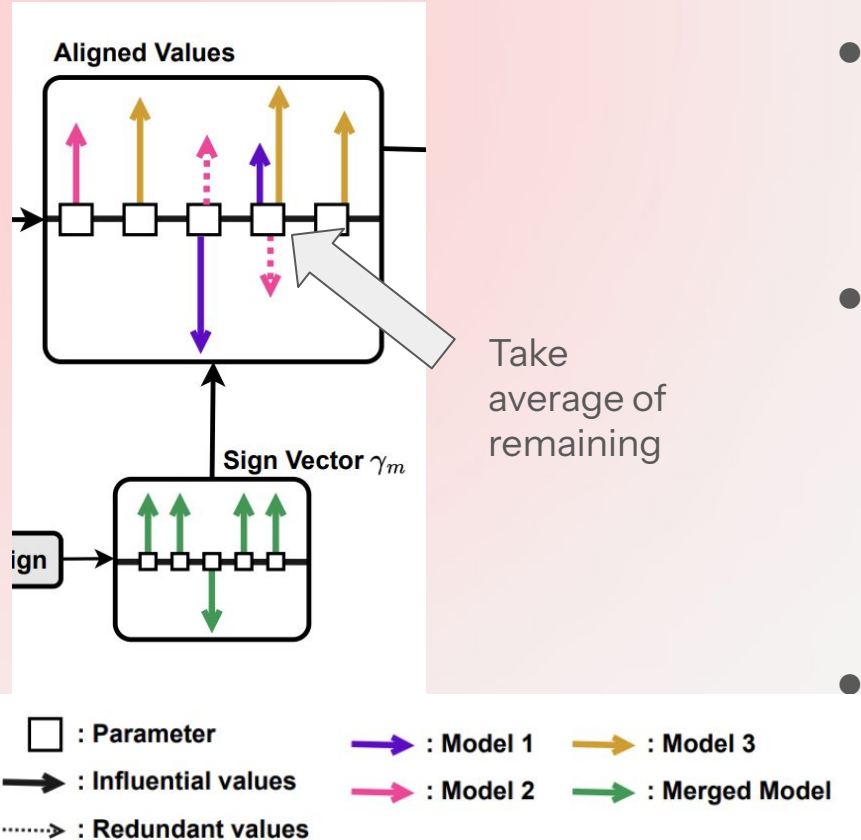


- After trimming step the trimmed task vectors remain
- Simply add the values, sign with highest magnitude is taken as sign
- Signs of each parameter are recorded in a sign vector (-1 or +1).



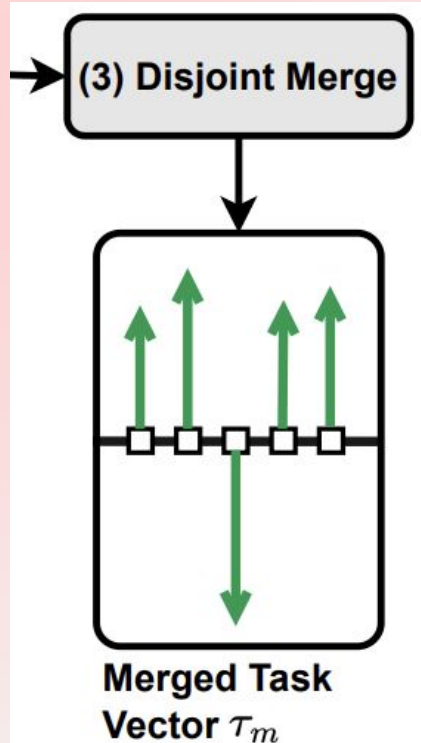
: Parameter
 : Influential values
 : Redundant values
 : Model 1
 : Model 2
 : Model 3
 : Merged Model

TIES Merging: Align



- Trimmed task vector is aligned with the sign vector.
- Remove all entries which are not aligned with the sign vector
 - The dotted arrows are meaningful delta weights whose signs do not match the sign vector.
- Remaining weights are then averaged for the final value.
 - E.g., parameter 4 has 2 weights after align, then take the mean of those.

TIES Merging: Merge



- For each remaining parameter value, take the mean, ignoring values set to zero
- Once the new task vector is created, they scale it by a merging coefficient factor λ which is a hyperparameter
- Now this merged task vector (model weights) can be added to the pretrained model initial parameters



Experimental Setup Baseline Methods

Methods:

1. Simple Averaging
 - a. Takes element-wise mean of finetuned model w/o weighting or sign resolution
2. Fisher Merging
 - a. Weights each parameter by how important it is to the task
3. RegMean
 - a. Solves least squares problem, finds merged weights that keep layer activations as close to each models activations
4. Task Arithmetic
 - a. Computes task vectors then scales them and sums them without trimming or sign resolution

Models:

1. T5-Base / T5-Large
 - a. Google's encoder-decoder transformer for NLP. Frames every task as text-in → text-out. Base ~220M params, Large ~770M.
2. ViT-B/32 and ViT-L/14
 - a. Vision Transformer. Splits images into patches and treats them as tokens. B=Base, L=Large; /32 and /14 are the patch sizes in pixels (smaller patch = more detail, more compute). These are CLIP visual encoders trained on image-text pairs.

Experimental Setup Baseline Methods

NLP Tasks

- **Sentence completion:** COPA, HellaSwag, Story Cloze
- **Natural language inference:** ANLI, CB, RTE — does sentence A entail, contradict, or neutral sentence B?
- **Coreference resolution:** WSC, Winogrande
- **Word sense disambiguation:** WiC — is this word used with the same meaning in two sentences?

Vision Tasks

- Cars: **classify car** make/model
- DTD: **texture classification** (is this image "bubbly", "striped", etc.)
- EuroSAT: **satellite image land use** (forest, highway, industrial, etc.)
- GTSRB: German **traffic sign recognition**
- MNIST: **handwritten digit classification**
- RESISC45: remote sensing **scene classification** (airport, beach, stadium, etc.)
- SUN397: **scene recognition** (kitchen, bedroom, highway, etc.)
- SVHN: street view **house number recognition**

Experimental Setup Baseline Methods

Methods:

1. No validation, no merging

- a. **Fine-Tuned:** Each task has its own dedicated model, no merging at all. Best possible performance per task.
- b. **Multitask:** trained jointly on all tasks at once from scratch.

2. No validation set used

- a. These methods **can't** tune hyperparameters or use data at merge time
- b. **Averaging:** just average all weights
- c. **Task arithmetic:** adds scaled task vectors
- d. **TIES merging:** principled method described in this paper

3. Validation set used

- a. These methods **can** tune hyperparameters or use data at merge time
- b. **Fisher Merging:** measures importance using a fisher similarity matrix, requires running data through model
- c. **RegMean:** needs to use layer activations, so has to use validation set to compute them
- d. **Task Arithmetic:** same as above, but uses validation set to tune λ
- e. **TIES merging:** same as above, but uses validation set to tune λ

Results: Merging PEFT, Vision, & NLP Models

Method (↓)	Validation	PEFT	Full Finetuning			
Model (→)		(IA) ³	T5-Base	T5-Large	ViT-B/32	ViT-L/14
FINE-TUNED	-	71.4	82.8	88.8	90.5	94.2
MULTITASK	-	73.1	83.6	88.1	88.9	93.5
AVERAGING [82, 9]	✗	-	65.9	59.6	65.8	79.6
TASK ARITHMETIC [29]	✗	-	73.2	73.5	60.4	83.3
TIES-MERGING	✗	-	69.7 [-3.2]	74.4 [+0.9]	72.4 [+6.6]	86.0 [+2.7]
FISHER MERGING [45]	✓	62.2	68.9	64.6	68.3	82.2
REGMEAN [31]	✓	58.0	71.2	73.2	71.8	83.7
TASK ARITHMETIC [29]	✓	63.9	73.2	73.3	70.1	84.5
TIES-MERGING	✓	66.4 [+2.5]	73.9 [+0.7]	76.9 [+3.6]	73.6 [+1.8]	86.0 [+1.5]

Table 1: Comparing model merging methods across multiple fine-tuning settings and modalities (NLP and Vision) with and without the availability of a validation set.

Results: Out-of-Domain Generalization

- They tested their merged models for NLP tasks on completely held out tasks that were never seen during fine tuning or merging.
 - Used zero shot as baseline
 - no fine tuning at all
 - Idea is that a good model should generalize
- Simple averaging and task arithmetic are barely above zero shot
- Data dependent methods help
- TIES is a clear winner

Model	T5-Base	T5-Large
Zeroshot	31.1	27.6
Simple Averaging [9, 82]	31.7	30.4
Fisher [45]	33.8	32.0
RegMean [31]	34.3	36.0
Task Arithmetic [29]	31.9	32.3
TIES-MERGING	35.3 [+1.0]	40.4 [+4.4]

Table 2: **TIES-MERGING generalizes better.** Out of Distribution Generalization for T5-Base and T5-Large on six held-out tasks.

Results: Merging Different Number of Tasks

- They checked different methods for merging increasing numbers of tasks/models.
- As the number of models increases, the performance for all drops (probably due to conflicts of parameters with number of models).
- Both TIES and Task Arithmetic achieve a normalized accuracy score close to one indicating negligible performance loss.

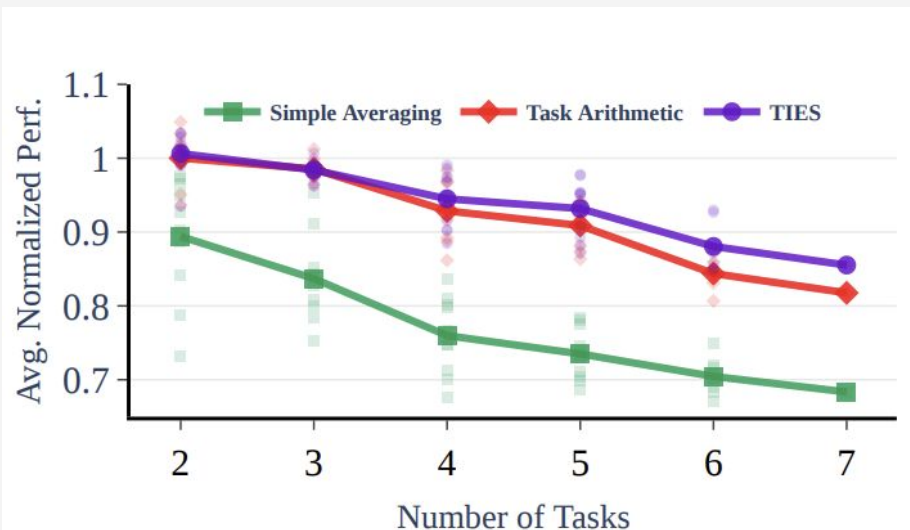


Figure 5: TIES-MERGING scales better. Average performance when merging a different number of tasks.

Results: Model Soups (Merging Checkpoints)

- Merge 10 different checkpoints all trained on the same task
 - TIES sign resolution should matter less
- 10 fine tuned BERT models from Hugging Face for 3 glue tasks
 - RTE (textual entailment)
 - MRPC (paraphrase detection)
 - WNLI (coreference/entailment)
- TIES still wins on RTE (72.2) and MRPC (86.8), beating even ensembling

	RTE	MRPC	WNLI
Averaging	59.9	78.2	56.3
Fisher	65.7	81.4	52.1
Ensembling	70.8	86.0	45.1
Task Arithmetic	71.8	86.0	59.2
TIES-MERGING	72.2	86.8	58.8

Table 3: Model soups experimental setup. TIES improves performance when merging checkpoints on the same tasks. For each task, we merge 10 checkpoints from Huggingface hub and evaluate on the one task they were trained on.

Results: Merging Models for Better Initialization

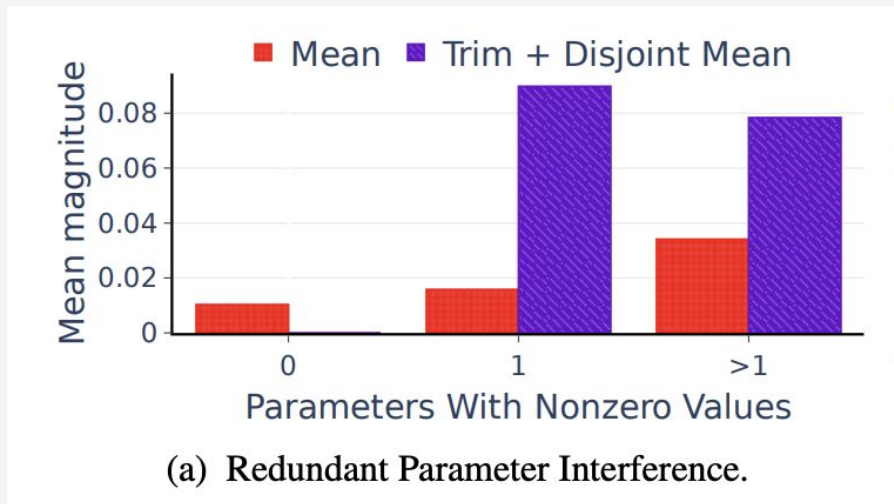
- Merged models and used these as starting point for fine tuning.
 - They took a BERT fine-tuned on 8 GLUE tasks, picked one as the target, merged all 7 together, then used that merged model as the initialization and fine tuned it on the target task.
- PTM Init: just start from the pretrained BERT, no merging at all
- Average: merge the 7 others by averaging, then fine-tune
- Task Arithmetic: merge via task vectors, then fine-tune
- TIES: merge via TIES, then fine-tune

Init Method	RTE	MRPC	WNLI
PTM Init	66.4	81.8	56.3
Average	75.8	86.5	56.3
Task Arithmetic	78.3	86.2	50.7
TIES-MERGING	80.1	88.0	54.9

Table 4: **A TIES-merged model is a better initialization for finetuning.** For each task, we merge the checkpoints from the 7 other GLUE tasks and then finetune and evaluate on the selected task.

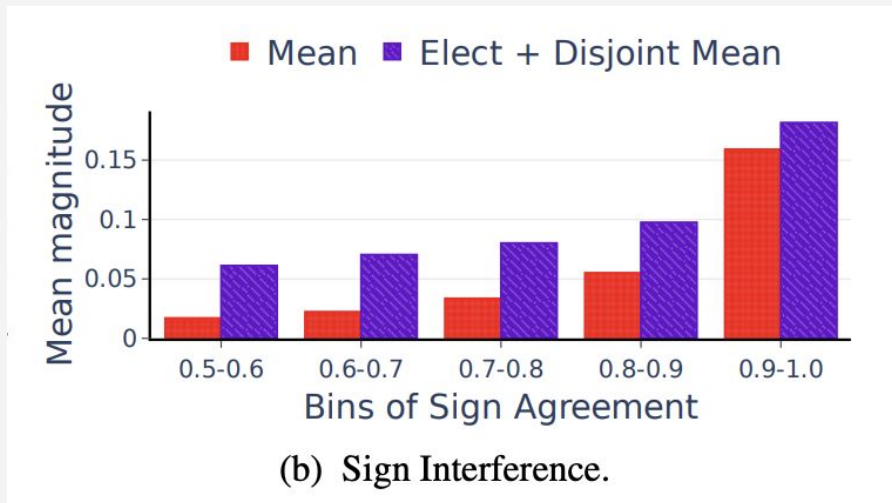
Results: Importance of Removing Redundant Parameters

- Why does trimming help, what does redundant parameter interference actually look like in the weights.
- Split all parameters into 3 groups:
 - Redundant — bottom 80%, trimmed to zero (not influential for any task)
 - Influential for exactly 1 model — only one task cares about this parameter
 - Influential for more than 1 model — multiple tasks care about this parameter
- Compared merging with and without trimming



Results: Results: Importance of Resolving Sign Interference

- Grouped parameters by how much the models agree on their sign
 - 0.5: perfect split half positive half negative
 - 1.0: all models agree on the same sign
- Compared merging with a plain mean vs. electing the sign first then disjoint mean
- Without sign election, parameters with low sign agreement get dragged toward zero (cancel each other out)
- With sign election, this is avoided so the magnitude is preserved



Results: Relevance of Signs of the Top-k% Parameters

- Took the top percentages of highest magnitude delta weights and flipped the signs of the PEFT IA³ (NLP) model and added back to the base model.
- Found that the direction of the top parameters are very important and more frequent sign flipping lead to catastrophic performance issues.
- Sign flipping of least important parameters didn't affect performance much at all.

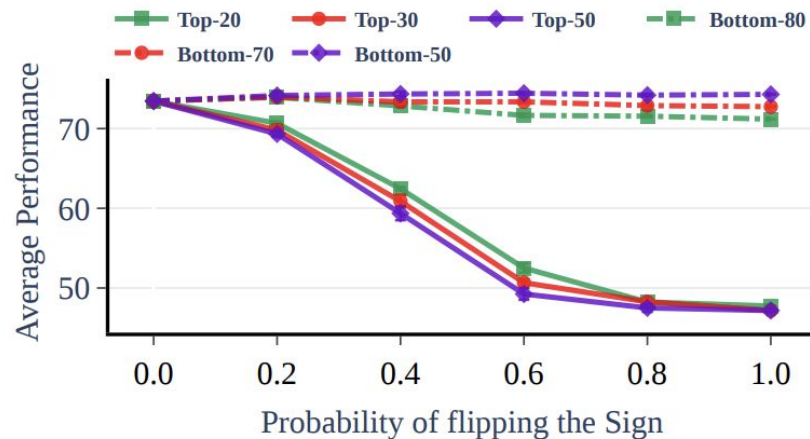


Figure 7: **Flipping the signs of high magnitude parameters leads to catastrophic performance drops.** Average Performance when flipping the directions of Top- $k\%$ and Bottom- $k\%$ parameters for each task. We report the results averaged over eleven (IA)³ tasks.

Conclusions

- Overall, TIES merging though simple and somewhat intuitive, seems to perform fairly well for merging model weights.
- It might still be unclear as to why such entanglement of model weights occur and how their relationships work.
- Another caveat is that these methods still tend to lag behind joint/simultaneous training in performance.



Questions?

